AIPoPh Internal Audit

The aim of this internal audit is to conduct thorough system testing on AIPoPh to ensure it functions as intended, identify any potential vulnerabilities, analyze performance across various scenarios, assess system robustness, and verify compliance with privacy regulations. The audit will result in a detailed report outlining our findings, proposed strategies for addressing any issues uncovered, and potential mitigations.

Findings

The findings of our internal audit is below in no particular order.

IA-1: ***All single validators need to switch from `when is` into `expect`***

Severity: Low Risk

Status:

Files Affected: All Validators

Description: All of the validators destruct the script context purpose with a `when is` statement causing redundant code that increases execution cost.

Recommendation: Change validators in this format:

```
when context.purpose is {
    Spend(output_reference) -> {..}
    _ -> False
}
```

into this format

```
expect Spend(output_reference) = context.purpose
..
```

The `expect` keyword should be used since it will auto fail instead of returning `False`.

IA-2: ***Transaction context should be destructured instead of using `.access`***

Severity: Low Risk

Status:

Files Affected: All Validators

Description: The transaction context is an expensive data structure to reference many times. If the validator references the Transaction type once then using the `.access` method is valid. If many fields need to be accessed from the Transaction type then the transaction context should be destructured.

Recommendation: Change the transaction context from this

```
let tx: Transaction = context.transaction
..
```

into this

```
let Transaction {
  inputs,
  outputs,
  ..
} = context.transaction
```

This greatly reduces execution costs when traces are turned off.

IA-3: ***Change dao validator into reference validator***

Severity: Informational

Status:

Files Affected: dao.ak

Description: The DAO contract is not a dao but rather just a reference data contract used to store data required for AIPoPh to operate. The naming convention is confusing.

Recommendation: Change dao into reference for the validator, type, and scripts. The DAO token used to perform actions should have a name that reflects the purpose of the token, like threshold token, vote token, action token, etc. The reference.ak file should be renamed to query.ak since it contains functions for querying the reference data.

IA-4: ***Remove explicit Lovelace addition to values when datum change***

Severity: Low Risk

Status:

Files Affected: dao.ak

Description: The explicit use of a Lovelace term inside the redeemer is redundant. It increases complexity and cost without gaining any security benefits.

Recommendation: Use `value.without_lovelace` on this value from the dao contract then use an `payout.at_least` to allow for the minimum required Lovelace to scale naturally.

IA-5: ***Remove dRep system entirely from the AIPoPh ecosystem***

Severity: High Risk

Status:

Files Affected: dao.ak, oracle.ak, vault.ak, poh_mint.ak, coh_mint.ak

Description: The dRep system is not defined in the intial scope and adds too much complexity to the system. The current implementation of the dRep system is incomplete and will cause security issues if allowed to stay inside the ecosystem at this time.

Recommendation: The contracts will still need to have a single input with a specific script bypass but without the drep contract. The count function should be counting by address which will allow for two different contracts to be spent as intended.

IA-6: ***Vault should allow adding and removing Tokens instead of a Token***

Severity: Low Risk

Status:

Files Affected: vault.ak

Description: The vault contract only allows a single Token to be added or removed from the UTxO. This limits functionality and will cause an increase to cost when adding or removing many tokens from the UTxO.

Recommendation: Switch the Token type into Tokens, and make sure all the Tokens are adding or subtracting. This will cover the single token case and improve functionality.

IA-7: ***Add documentation to utility functions inside the library***

Severity: Informational

Status:

Files Affected: types/prove.ak, types/graph.ak, types/coh.ak, types/dao.ak types/poh.ak reference.ak

Description: Documentation is missing from key functions inside various files. These utility functions need proper documentation to describe their purpose and use case.

Recommendation: Add in some `///` top level function comments to all public functions.

IA-8: ***Potential stuck UTxO during VerifyTest redeemer due to lack of gas***

Severity: High Risk

Status:

Files Affected: poh_lock.ak

Description: If a user does not place enough Lovelace onto the test UTxO then during the VerifyTest state where a threshold actor comes to verify the test taken by the user. The UTxO may become perma stuck due to the validation logic of Quit demanding that the verification process finish before removing.

Recommendation: A simple fix is making any premature leave of the test result in a lost deposit as if the user failed the test. Then not having enough gas will just throw the UTxO into the Quit state with a lost deposit and not a perma lock.

IA-9: ***Potential collusion attack with the EndTest redeemer due to trust of web signer***

Severity: High Risk

Status:

Files Affected: poh_lock.ak

Description: A trusted website must host the graph coloring test. The site itself has to sign off on the data for the Answer type to be valid. If the web signer signs incorrect data, fails to sign the data, or refuses to sign the data then the test will always fail due to a bad web signer signature.

Recommendation: The website part of the test must be robust and fault tolerant to prevent this type of attack from occurring during valid tests. Failing a test means the user loses their deposit. If the Answer is valid but the web signer signature fails then either the user should get their deposit back or the website should have some off-chain contesting feature that would allow the user to contact the test company.

IA-10: ***Inability to prove that random data from the oracle is sufficently random***

Severity: High Risk

Status:

Files Affected: oracle.ak, poh_lock.ak

Description: The oracle requirement is that the new random data is different from the previous random data but it does not prevent reusing old data nor does it prove that the data itself is truely random. A bad oracle could reuse the same random values from a set of 2 or more values. This would generate a set of graphs of the same size as the number of unique random numbers. This could potentially make the coloring of these graphs trival, breaking the np-hardness of the coloring problem.

Recommendation: A merkle-tree of values may be applicable but randomness may be reoccurring. Verifiable randomness would be nice but that primative is not available.

IA-11: ***Destruct reference.get_data into two functions***

Severity: Low Risk

Status:

Files Affected: All Validators

Description: There is not benefit to writing this function to return a tuple.

```
// get the dao reference utxo
let (dao_datum, dao_input): (DAODatum, Input) = reference.get_data(this_tx.reference_inputs,
```

The dao_data_hash is not data either. It can be safely assumed to be a ValidatorHash. Which means the expect statement inside get_data is redundant.

```
expect dao_hash: ValidatorHash = dao_hash_data
```

This is expecting something to be in that type when it already is that type at the start.

Recommendation: Deflat the function back into its parts.

```
let dao_addr: Address = credential.from_script(dao_hash)
let dao_input: Input = find.input_by_addr(reference_inputs, dao_addr)
expect dao_datum: DAODatum = data.input_datum(dao_input)
```

IA-12: ***All validators need comments describing the validation above the `and` where applicable***

Severity: Informational

Status:

Files Affected: All Validators

Description: It is very convienient to have a quick description above the `and{..}` statement inside a validator using simple language. Some validator have it but others don't.

Recommendation: Add a couple of lines of documentation above the `and` statements in each validator describe what the and logic does for the validation.

IA-13: ***Write Extensive tests for Moment calculations***

Severity: Medium Risk

Status:

Files Affected: poh_lock.ak

Description: The Moment type is used extensively during the proof of humanity test phase. Any issues associated with the Moment type will transfer to issues with the test resulting in lost deposits.

Recommendation: Write tests that prove that the Moment type works as expected that go well beyond the Assist library tests.

IA-14: ***Add Top Level comments to the validators***

Severity: Informational

Status:

Files Affected: All Validators

Description: Write top level comments using `////` at the top of each validator that describes what the validator does and the point of it.

Recommendation: Some validator already have them so the ones that do have them need to be reviewed.

IA-15: ***Some public functions should be private and not exposed***

Severity: Low Risk

Status:

Files Affected: All Validators

Description: Some functions are defined to be public but they are only used as private helper functions. Find all public functions and make sure that they are actually supposed to be public else convert them to private functions.

Recommendation: Remove `pub` from functions that are really just private.

IA-16: ***Remove all functions that are not used in the code base***

Severity: Informational

Status:

Files Affected: All Validators

Description: There are functions that are defined but are never used anywhere.

Recommendation: Find and remove all unused functions.

IA-17: ***Proving the vault is being spent has redundant logic***

Severity: Low Risk

Status:

Files Affected: poh_mint.ak

Description: When losing a deposit, the vault must be spent along with the burning of the poh token. The logic does a redundant self check as shown below.

```
let is_vault_being_spent: Bool =
  when vault_redeemer is {
    AddToVault { incoming } -> incoming == incoming
    _ -> False
  }
```

Proving that the incoming variable is equal to itself is redundant. This `when is` statement can be simplified into an `expect` such that the code will auto fail if the wrong redeemer is used.

Recommendation: Either change the logic to just returning a true or remove the `when is` and replace it with an `expect` statement. The expect statement soluiton will be cheaper than simply returning a boolean.

IA-18: ***Replace PoHDatum integer state with token mint***

Severity: Medium Risk

Status:

Files Affected: poh_lock.ak, poh_mint.ak

Description: The StartTest redeemer does verify that the `datum.test_data.cur_stage` has the correct value. Holding state as an integer inside the datum is not recommended. State management should be governed by token possession.

Recommendation: Remove cur_stage from the datum and replace it with a proof of holding exactly a nft validation function. The cur_stage is just indication that the test has moved from the on going test phase to the end test phase. The logic itself must be reviewed as it may be possible to remove this all together.

IA-19: ***Petition threshold protection against collusion attacks***

Severity: High Risk

Status:

Files Affected: dao.ak

Description: The only requirement for the petition threshold is that it has to be greater than zero. There does not exist any protections against increasing it or decreasing it to dangerous values. For example, reducing the threshold to 1 to allow anyone to petition resulting in potentially easily revoked coh tokens. Or the inverse, increasing it to a value larger than the total token supply resulting in an unupdatable datum.

Recommendation: Review and investigate potential petition threshold restrictions. Knowledge of the dao token is limited to the policy id and token name. The total amount, contract closure, and distribution are hidden from the validator.

IA-#: **

Severity:

Status:

Files Affected: All Validators

Description:

Recommendation:

Conclusion

- TODO